

## **Assignment 2 – Event Handling [Part 1]**

### ***Goal***

To put into practice the event driven development principles and techniques covered in class, including function pointers, event handlers and callbacks. The goal of this assignment is to implement an event handling system for game development. This framework is a fundamental building block of modern game architectures, so it is important that all students gain some experience with its internal design and implementation.

### ***Description***

The event handling system implemented will support the two different approaches discussed in class: an event system based on callback functions and a generic event handler using templates. These systems are independent each other. For compatibility the name of the classes and some methods have been changed. The objective as the previous assignments is to get familiar with their implementations and create the foundations to apply event handling on a videogame. In this sense, it is necessary to full fill the following requirements:

#### **1. Base Event class (event.hh)**

You have to implement an empty abstract class **Event**. All Events must inherit from this class.

#### **2. Event system based on callbacks (event\_callbacks.hh):**

Callback function:

You have to implement a function pointer to handle all events. The name of the callback function datatype must be **CallbackFunction**.

EventListener: Object containing the who and the how

| Attributes       | Description                                               |
|------------------|-----------------------------------------------------------|
| Context          | Object interested in the event                            |
| CallbackFunction | Callback function to be executed once the event is raised |

EventDispatcher: Class that registers the objects listening the events and their callback function. It stores and notifies of an event to the listeners.

| Attributes              | Description                                                              |
|-------------------------|--------------------------------------------------------------------------|
| Listeners<br>Subscribed | Array or container of listeners subscribed to the events                 |
| Methods                 | Description                                                              |
| subscribe               | Allows listener objects to be registered with his static callback method |
| getListenerAt           | Allows to get a listener depending on its position in the array          |
| dispatchEvent           | Notifies all the listener that an event has happened                     |

### 3. Generic event handling (event\_handlers.hh):

IEventHandler: Interface that all Event handlers must implement

| Methods | Description                                |
|---------|--------------------------------------------|
| call    | Method that receives and handles the event |

EventHandler: Generic event handler class implemented using templates, requires to type names: **T** for the type of the object interested in the event and **E** for the type of the specific event.

| Attributes   | Description                                                                                                                                                                                              |
|--------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Object T     | Object of type <b>T</b> interested in the Event                                                                                                                                                          |
| ActionMethod | Function pointer to the method of the instance handling the event. It should receive as parameter an event of type <b>E</b> .                                                                            |
| Methods      | Description                                                                                                                                                                                              |
| constructor  | Constructor of the event handler class. It should receive a pointer to an object <b>T</b> interested in the event and the function pointer to the method in the class handling the event (ActionMethod). |
| call         | Implements the call method from the parent interface. It should execute the function pointer ActionMethod in the object of type <b>T</b> , passing an event of type <b>E</b> as parameter.               |

EventManager: Registers all the event handlers listening to the event and notifies them when an event happens.

| Attributes           | Description                                                            |
|----------------------|------------------------------------------------------------------------|
| Listeners Subscribed | Array or container of event handlers subscribed to the events          |
| Methods              | Description                                                            |
| registerEventHandler | Allows event handlers to be registered with his static callback method |
| getEventHandlerAt    | Allows to get an event handler depending on its position in the array  |
| notifyEvent          | Notifies all the event handlers that an event has happened             |

The primary objective of this first part is to ensure that all tests compile and pass successfully. Code quality, documentation, and overall organization will also be evaluated. Students should not modify any files related to the tests; only the files specified for the assignment can be changed. Upon completion, students should have implemented a usable framework capable of handling events.

***What to do:***

The code explicitly requires the following files, which the student should create and submit:

- event.hh, event\_callbacks.hh, event\_callbacks.cc, event\_handlers.hh and event\_handlers.cc

***Submission Guidelines***

Submit all the files as a zip folder on Moodle with the following naming conventions:

cs225fa26\_{username}\_2.1.zip